WAP to Implement 3D Transformations - Translation, Rotation and Scaling

```
#include <stdio.h>
#include <conio.h>
#include <graphics.h>
#include <math.h>

#define PI 3.14159

void drawcube(float x[], float y[], float z[])
{
   int i;
   int px[8], py[8];

   /* 3D to 2D projection */

   for(i = 0; i < 8; i++)
   {
      px[i] = (int)(x[i] + z[i] / 2);
      py[i] = (int)(y[i] - z[i] / 2);
   }

   /* Front face */

   line(px[0], py[0], px[1], py[1]);
   line(px[1], py[1], px[2], py[2]);
   line(px[2], py[2], px[3], py[3]);
   line(px[3], py[3], px[0], py[0]);

   /* Back face */

   line(px[4], py[4], px[5], py[5]);
   line(px[5], py[5], px[6], py[6]);
   line(px[6], py[6], px[7], py[7]);
   line(px[7], py[7], px[4], py[4]);

   /* Connecting edges */

   line(px[0], py[0], px[4], py[4]);
   line(px[1], py[1], px[5], py[5]);
   line(px[2], py[2], px[6], py[6]);
   line(px[3], py[3], px[7], py[7]);
}

void main()
{
   int gd = DETECT, gm;
   int choice, i;
```

```
float x[8] = {
   200, 300, 300, 200,
   200, 300, 300, 200
};

float y[8] = {
   180, 180, 280, 280,
   180, 180, 280, 280
};

float z[8] = {
   0, 0, 0, 0,
   80, 80, 80, 80
};

float nx[8], ny[8], nz[8];

float tx, ty, tz;
float sx, sy, sz;
float angle, rad;

float cx = 250;
float cy = 230;
float cz = 40;

initgraph(&gd, &gm, "C:/TURBO C3/BGI");

/* Draw original cube */

setcolor(WHITE);
drawcube(x, y, z);

printf("1. Translation\n");
printf("2. Rotation\n");
printf("3. Scaling\n");

printf("Enter your choice: ");
scanf("%d", &choice);

switch(choice)
{
   case 1:

      /* Translation */

      printf("Enter tx ty tz: ");
      scanf("%f%f%f", &tx, &ty, &tz);

      for(i = 0; i < 8; i++)
      {
```

```
        nx[i] = x[i] + tx;
        ny[i] = y[i] + ty;
        nz[i] = z[i] + tz;
    }

    break;

case 2:

    /* Rotation about Z-axis */

    printf("Enter rotation angle: ");
    scanf("%f", &angle);

    rad = angle * PI / 180.0;

    for(i = 0; i < 8; i++)
    {
        nx[i] = cx
            + (x[i] - cx) * cos(rad)
            - (y[i] - cy) * sin(rad);

        ny[i] = cy
            + (x[i] - cx) * sin(rad)
            + (y[i] - cy) * cos(rad);

        nz[i] = z[i];
    }

    break;

case 3:

    /* Scaling */

    printf("Enter sx sy sz: ");
    scanf("%f%f%f", &sx, &sy, &sz);

    for(i = 0; i < 8; i++)
    {
        nx[i] = cx + (x[i] - cx) * sx;
        ny[i] = cy + (y[i] - cy) * sy;
        nz[i] = cz + (z[i] - cz) * sz;
    }

    break;

default:
    printf("Invalid Choice");
    getch();
```

```
        closegraph();
        return;
    }

    /* Draw transformed cube */

    setcolor(YELLOW);
    drawcube(nx, ny, nz);

    getch();
    closegraph();
}
```